

Design Document

NightOUT

Version: 0.1

Authors: Riccardo Masarin, Manuel Mannucci, Alessandro Forlani

Project: Software Engineering for Automation — Project A

Table of Contents

1. Introduction
 - 1.1 Purpose
 - 1.2 Definitions, Acronyms and Abbreviations
 - 1.3 Document Structure
2. Architectural Design
 - 2.1 Overview
 - 2.2 Component View
 - 2.3 Class View
 - 2.4 Runtime View
 - 2.5 Selected Architectural Styles and Patterns
3. User Interface Design
 - 3.1 UI Overview
 - 3.2 Main User Interfaces
 - 3.3 Venue Manager Interface
4. Requirements Traceability
 - 4.1 Traceability Matrix
 - 4.2 Traceability Discussion
5. Implementation, Integration and Test Plan
 - 5.1 Implementation Order
 - 5.2 Integration Plan
 - 5.3 Testing Strategy
 - 5.4 Test Data
6. References

1. Introduction

1.1 Purpose

The purpose of this document is to describe the design of **NightOUT**, a mobile-oriented web application for nightlife discovery, social interaction, pregame coordination, and reservation management.

This document refines the requirements defined in the Requirements Analysis and Specification Document and describes how the system will be structured and implemented. In particular, it presents the software architecture, the main components, the relevant class diagrams, the runtime interactions among components, the user interface design, the mapping between requirements and design elements, and the implementation, integration, and testing plan.

The design of NightOUT aims to support the following goals:

- provide a modular and maintainable architecture;
- separate presentation, business logic, domain logic, and persistence;
- make the backend implementation directly traceable to the UML design;
- support the main design patterns selected for the project;
- allow incremental implementation and testing of the prototype;
- provide a mobile-first user experience while keeping frontend complexity manageable.

NightOUT will be developed as a **mobile-oriented responsive web application**. The backend will be implemented in **Java using Spring Boot**, while the frontend will be implemented as a **React web application optimized for mobile usage**. The backend exposes REST APIs and contains the main business logic, while the frontend provides the user interface for event discovery, reservation, social interaction, pregame rooms, and venue manager operations.

1.2 Definitions, Acronyms and Abbreviations

NightOUT

The software system described in this document.

User

A registered person using the system to discover events, reserve spots, join pregame rooms, and interact with social features.

Venue Manager

A verified account responsible for managing venues, publishing events, defining promotions, and viewing basic statistics.

Venue

A physical place, such as a club, bar, lounge, or live music venue, where nightlife events take place.

Event

A nightlife activity associated with a venue.

Reservation

A digital record representing a user request to attend an event.

Waiting List

A list of users waiting for a spot in a full event.

Pregame Room

A temporary group linked to an event, used by users to coordinate before attending the event.

Notification

A message generated by the system to inform a user about relevant updates.

REST API

Representational State Transfer Application Programming Interface.

DTO

Data Transfer Object.

UI

User Interface.

DD

Design Document.

1.4 Document Structure

Section 1 introduces the purpose, definitions, revision history, and structure of the document.

Section 2 presents the architectural design of the system, including component view, class view, runtime view, and selected architectural styles and design patterns.

Section 3 describes the user interface design and the main screens of the application.

Section 4 provides the requirements traceability matrix, showing how the requirements defined in the Requirements Analysis and Specification Document are mapped to design elements.

Section 5 describes the implementation, integration, and testing plan.

Section 6 lists the references used in the document.

2. Architectural Design

2.1 Overview

NightOUT follows a **client-server architecture**.

The client side is a **React mobile-first web frontend**. It provides the user interface and communicates with the backend through REST API calls.

The server side is a **Java Spring Boot backend**. It implements the application logic, domain logic, design patterns, persistence coordination, and REST endpoints.

The database stores users, venue managers, venues, events, reservations, pregame rooms, promotions, notifications, social relations, and event participation information.

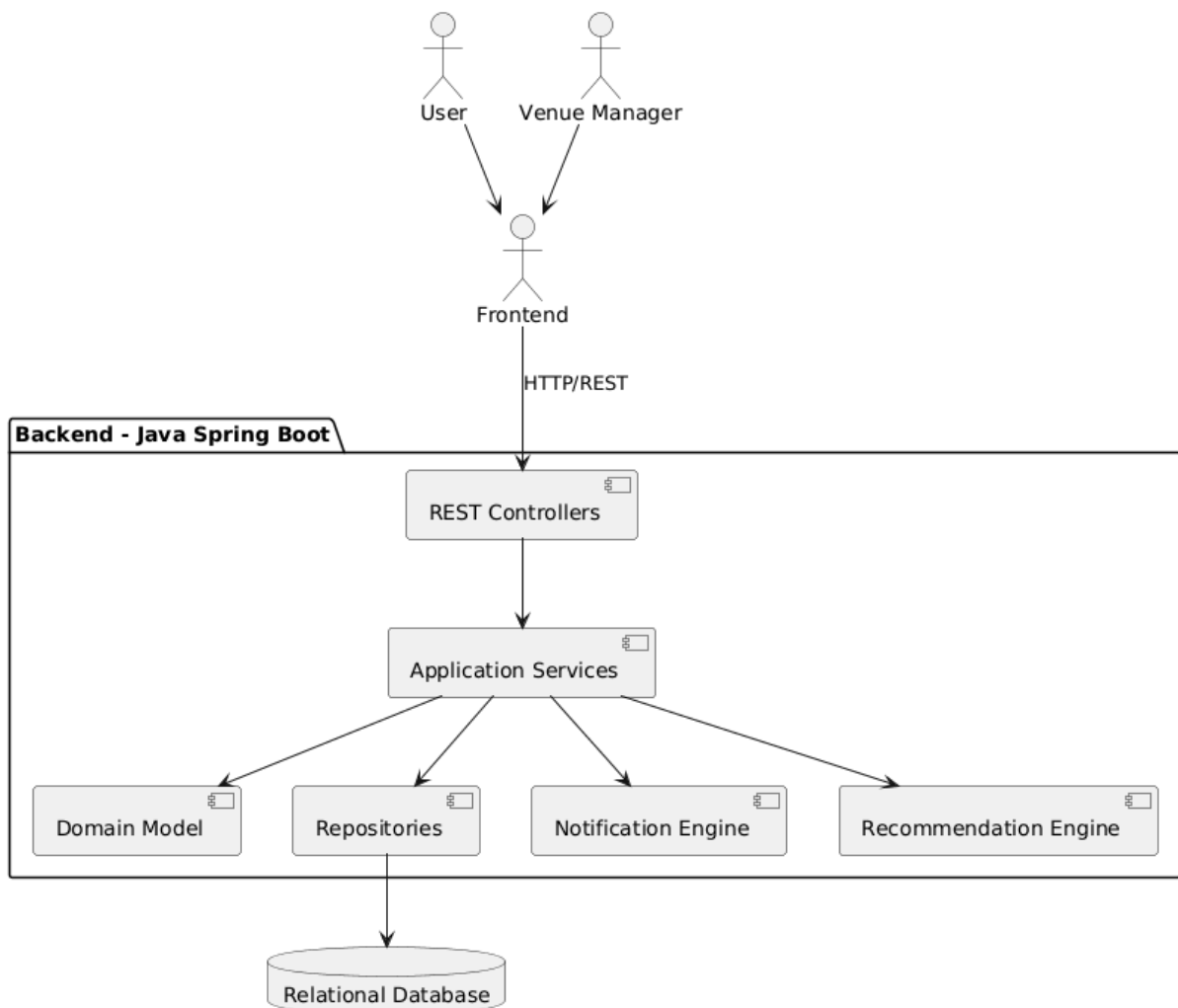
The system uses a **layered architecture** inside the backend:

1. **Controller layer**
Exposes REST APIs to the frontend.
2. **Service layer**
Implements application logic and coordinates domain operations.
3. **Domain layer**
Contains the core business entities and domain behavior.
4. **Repository layer**
Provides access to persistent data.
5. **Database layer**
Stores application data.

This design separates concerns and allows the backend to remain modular, testable, and maintainable.

The general architecture is shown below.

NightOUT - High-Level Architecture



@startuml

title NightOUT - High-Level Architecture

actor User

actor "Venue Manager" as VenueManager

```

package "Frontend" {
    component "React Mobile-First Web App" as Frontend
}
  
```

```

package "Backend - Java Spring Boot" {
    component "REST Controllers" as Controllers
    component "Application Services" as Services
    component "Domain Model" as Domain
    component "Repositories" as Repositories
    component "Notification Engine" as NotificationEngine
  
```

```
component "Recommendation Engine" as RecommendationEngine
}

database "Relational Database" as DB

User --> Frontend
VenueManager --> Frontend

Frontend --> Controllers : HTTP/REST

Controllers --> Services
Services --> Domain
Services --> Repositories
Services --> NotificationEngine
Services --> RecommendationEngine

Repositories --> DB

@enduml
```

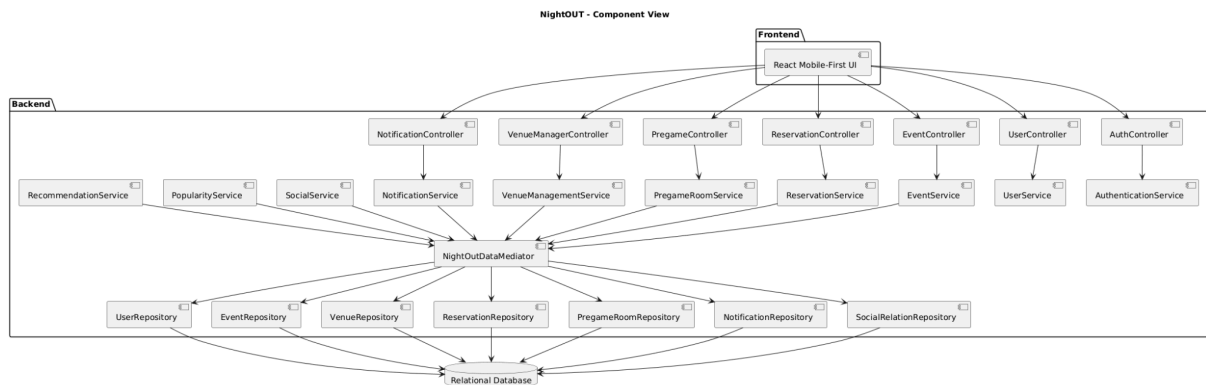
2.2 Component View

The component view describes the main software components and the services they provide.

NightOUT is decomposed into the following components:

1. **Authentication and User Management Component**
2. **Venue Management Component**
3. **Event Management Component**
4. **Event Search and Filtering Component**
5. **Reservation and Ticketing Component**
6. **Pregame Room Component**
7. **Social Interaction Component**
8. **Notification Component**
9. **Popularity and Recommendation Component**
10. **Data Access Component**

The component diagram is shown below.



@startuml

title NightOUT - Component View

```

package "Frontend" {
  component "React Mobile-First UI" as UI
}
  
```

```

package "Backend" {
  component "AuthController" as AuthController
  component "UserController" as UserController
  component "EventController" as EventController
  component "ReservationController" as ReservationController
  component "PregameController" as PregameController
  component "VenueManagerController" as VenueManagerController
  component "NotificationController" as NotificationController
  
```

```

  component "AuthenticationService" as AuthService
  component "UserService" as UserService
  component "EventService" as EventService
  component "ReservationService" as ReservationService
  component "PregameRoomService" as PregameService
  component "VenueManagementService" as VenueService
  component "NotificationService" as NotificationService
  component "SocialService" as SocialService
  component "PopularityService" as PopularityService
  component "RecommendationService" as RecommendationService
  
```

```

  component "NightOutDataMediator" as DataMediator
  
```

```

  component "UserRepository" as UserRepo
  component "EventRepository" as EventRepo
  component "VenueRepository" as VenueRepo
  component "ReservationRepository" as ReservationRepo
  component "PregameRoomRepository" as PregameRepo
  
```

```
component "NotificationRepository" as NotificationRepo
component "SocialRelationRepository" as SocialRepo
}
```

```
database "Relational Database" as DB
```

```
UI --> AuthController
UI --> UserController
UI --> EventController
UI --> ReservationController
UI --> PregameController
UI --> VenueManagerController
UI --> NotificationController
```

```
AuthController --> AuthService
UserController --> UserService
EventController --> EventService
ReservationController --> ReservationService
PregameController --> PregameService
VenueManagerController --> VenueService
NotificationController --> NotificationService
```

```
EventService --> DataMediator
ReservationService --> DataMediator
PregameService --> DataMediator
VenueService --> DataMediator
NotificationService --> DataMediator
SocialService --> DataMediator
PopularityService --> DataMediator
RecommendationService --> DataMediator
```

```
DataMediator --> UserRepo
DataMediator --> EventRepo
DataMediator --> VenueRepo
DataMediator --> ReservationRepo
DataMediator --> PregameRepo
DataMediator --> NotificationRepo
DataMediator --> SocialRepo
```

```
UserRepo --> DB
EventRepo --> DB
VenueRepo --> DB
ReservationRepo --> DB
PregameRepo --> DB
NotificationRepo --> DB
SocialRepo --> DB
```

2.2.1 Authentication and User Management Component

This component manages user registration, login, user profiles, and music preferences.

Main services:

- register user;
- authenticate user;
- retrieve user profile;
- update user preferences;
- retrieve saved events and reservations.

2.2.2 Venue Management Component

This component manages venues and venue manager operations.

Main services:

- create venue;
- update venue;
- retrieve venues managed by a venue manager;
- create and update events;
- create and update promotions;
- retrieve basic event statistics.

2.2.3 Event Management Component

This component manages event information.

Main services:

- create event;
- update event;
- retrieve event details;
- retrieve events by venue;
- retrieve available events.

2.2.4 Event Search and Filtering Component

This component allows users to search and filter events according to selected criteria.

Main services:

- filter by date;
- filter by location;

- filter by music genre;
- filter by venue category;
- filter by price or entry condition.

The filtering logic is implemented using the Strategy pattern.

2.2.5 Reservation and Ticketing Component

This component manages reservation requests, reservation states, event capacity, waiting lists, and cancellation.

Main services:

- request reservation;
- confirm reservation;
- place reservation in waiting list;
- cancel reservation;
- promote waiting list reservation;
- retrieve reservation status;
- retrieve user tickets.

The reservation lifecycle is implemented using the State pattern.

2.2.6 Pregame Room Component

This component manages pregame rooms linked to events.

Main services:

- create pregame room;
- retrieve pregame rooms by event;
- join pregame room;
- leave pregame room;
- check room capacity;
- retrieve room participants.

2.2.7 Social Interaction Component

This component manages social relations and event participation.

Main services:

- create social relation;
- retrieve friends or followed users;
- save event;
- set participation status;
- retrieve friends attending an event;
- generate activity feed data.

2.2.8 Notification Component

This component creates and manages notifications.

Main services:

- create notification;
- retrieve user notifications;
- mark notification as read;
- notify reservation status change;
- notify waiting list promotion;
- notify friend event participation;
- notify event updates.

The notification logic is implemented using the Observer pattern.

2.2.9 Popularity and Recommendation Component

This component computes event popularity and generates recommended events.

Main services:

- compute event popularity score;
- order events by selected popularity criteria;
- recommend events based on music preferences;
- recommend events based on social proximity;
- recommend events based on distance;
- recommend events based on booking history.

Popularity and recommendation algorithms are implemented using the Strategy pattern.

2.2.10 Data Access Component

This component encapsulates persistence access through repositories. A mediator-like component, `NightOutDataMediator`, coordinates complex data access operations involving multiple repositories.

Main services:

- retrieve users;
- retrieve venues;
- retrieve events;
- retrieve reservations;
- retrieve pregame rooms;
- retrieve notifications;
- retrieve social relations;
- save domain entities;
- coordinate multi-entity queries.

```
class User {
    - id: Long
    - name: String
    - email: String
    - passwordHash: String
    - musicPreferences: List<MusicGenre>
    + saveEvent(event: Event): void
    + updatePreferences(preferences: List<MusicGenre>): void
}
```

```
class VenueManager {  
  - id: Long  
  - name: String  
  - email: String  
  - verified: Boolean  
  + createEvent(eventData: EventData): Event  
  + updateEvent(event: Event): void  
}
```

```
class Venue {  
  - id: Long  
  - name: String  
  - category: VenueCategory  
  - address: String  
  - city: String  
  - area: String  
}
```

```
class Event {  
  - id: Long  
  - title: String  
  - dateTime: LocalDateTime  
  - musicGenre: MusicGenre  
  - dressCode: String  
  - entryCondition: String  
  - price: Double  
  - capacity: Integer  
  + hasAvailableCapacity(): Boolean  
}
```

```
class Reservation {  
  - id: Long  
  - status: ReservationStatus  
  - createdAt: LocalDateTime  
  - state: ReservationState  
  + confirm(): void  
  + cancel(): void  
  + expire(): void  
  + moveToWaitingList(): void  
}
```

```
class PregameRoom {  
  - id: Long  
  - meetingLocation: String  
  - meetingTime: LocalDateTime  
  - maxParticipants: Integer
```

```
- description: String
+ hasAvailableCapacity(): Boolean
+ addParticipant(user: User): void
+ removeParticipant(user: User): void
}
```

```
class Promotion {
- id: Long
- description: String
- validFrom: LocalDateTime
- validTo: LocalDateTime
}
```

```
class Notification {
- id: Long
- type: NotificationType
- message: String
- read: Boolean
- createdAt: LocalDateTime
+ markAsRead(): void
}
```

```
class SocialRelation {
- id: Long
- type: SocialRelationType
}
```

```
class EventParticipation {
- status: ParticipationStatus
}
```

```
enum ReservationStatus {
    PENDING
    CONFIRMED
    WAITING_LIST
    CANCELLED
    EXPIRED
}
```

```
enum MusicGenre {
    TECHNO
    HOUSE
    HIP_HOP
    POP
    COMMERCIAL
    LATIN
    ROCK
}
```

```
LIVE_MUSIC
}
```

```
enum VenueCategory {
    CLUB
    BAR
    LOUNGE
    LIVE_MUSIC_VENUE
}
```

```
enum NotificationType {
    RESERVATION_UPDATE
    FRIEND_JOINED_EVENT
    WAITING_LIST_AVAILABLE
    EVENT_UPDATE
    PREGAME_UPDATE
}
```

```
enum SocialRelationType {
    FRIENDSHIP
    FOLLOWING
}
```

```
enum ParticipationStatus {
    INTERESTED
    SAVED
    RESERVED
    ATTENDING
}
```

```
VenueManager "1" -- "1..*" Venue : manages
Venue "1" -- "0..*" Event : hosts
Event "1" -- "0..*" Reservation : has
User "1" -- "0..*" Reservation : owns
```

```
Event "1" -- "0..*" PregameRoom : linked to
User "1" -- "0..*" PregameRoom : hosts
User "0..*" -- "0..*" PregameRoom : joins
```

```
Event "1" -- "0..*" Promotion : offers
Venue "1" -- "0..*" Promotion : offers
```

```
User "1" -- "0..*" Notification : receives
```

```
User "1" -- "0..*" SocialRelation : source
SocialRelation "1" -- "1" User : target
```

User "1" -- "0..*" EventParticipation : has
EventParticipation "1" -- "1" Event : refers to

Reservation --> ReservationStatus

Event --> MusicGenre

Venue --> VenueCategory

Notification --> NotificationType

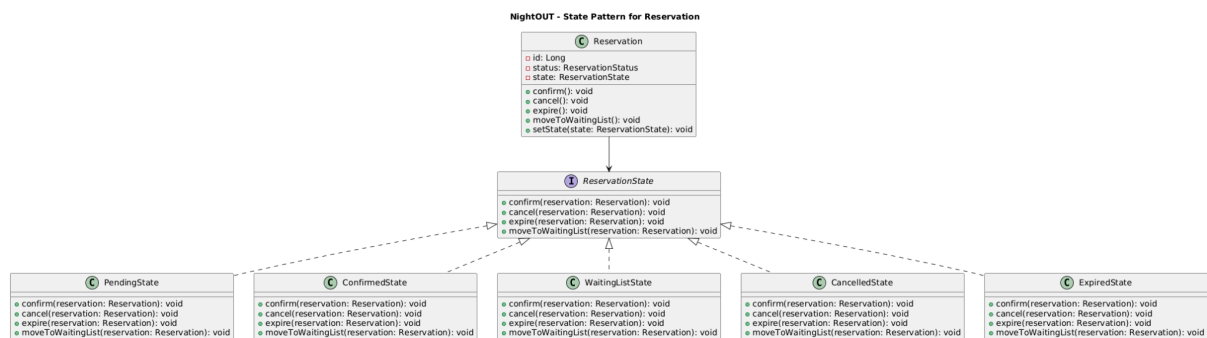
SocialRelation --> SocialRelationType

EventParticipation --> ParticipationStatus

@enduml

2.3.2 Reservation State Pattern Class Diagram

The State pattern is used to model the lifecycle of a reservation. The allowed operations depend on the current state of the reservation.



@startuml

title NightOUT - State Pattern for Reservation

```
interface ReservationState {
    + confirm(reservation: Reservation): void
    + cancel(reservation: Reservation): void
    + expire(reservation: Reservation): void
    + moveToWaitingList(reservation: Reservation): void
}
```

```
class Reservation {
    - id: Long
    - status: ReservationStatus
    - state: ReservationState
    + confirm(): void
    + cancel(): void
    + expire(): void
    + moveToWaitingList(): void
}
```

```

+ setState(state: ReservationState): void
}

class PendingState {
+ confirm(reservation: Reservation): void
+ cancel(reservation: Reservation): void
+ expire(reservation: Reservation): void
+ moveToWaitingList(reservation: Reservation): void
}

class ConfirmedState {
+ confirm(reservation: Reservation): void
+ cancel(reservation: Reservation): void
+ expire(reservation: Reservation): void
+ moveToWaitingList(reservation: Reservation): void
}

class WaitingListState {
+ confirm(reservation: Reservation): void
+ cancel(reservation: Reservation): void
+ expire(reservation: Reservation): void
+ moveToWaitingList(reservation: Reservation): void
}

class CancelledState {
+ confirm(reservation: Reservation): void
+ cancel(reservation: Reservation): void
+ expire(reservation: Reservation): void
+ moveToWaitingList(reservation: Reservation): void
}

class ExpiredState {
+ confirm(reservation: Reservation): void
+ cancel(reservation: Reservation): void
+ expire(reservation: Reservation): void
+ moveToWaitingList(reservation: Reservation): void
}

```

Reservation --> ReservationState

```

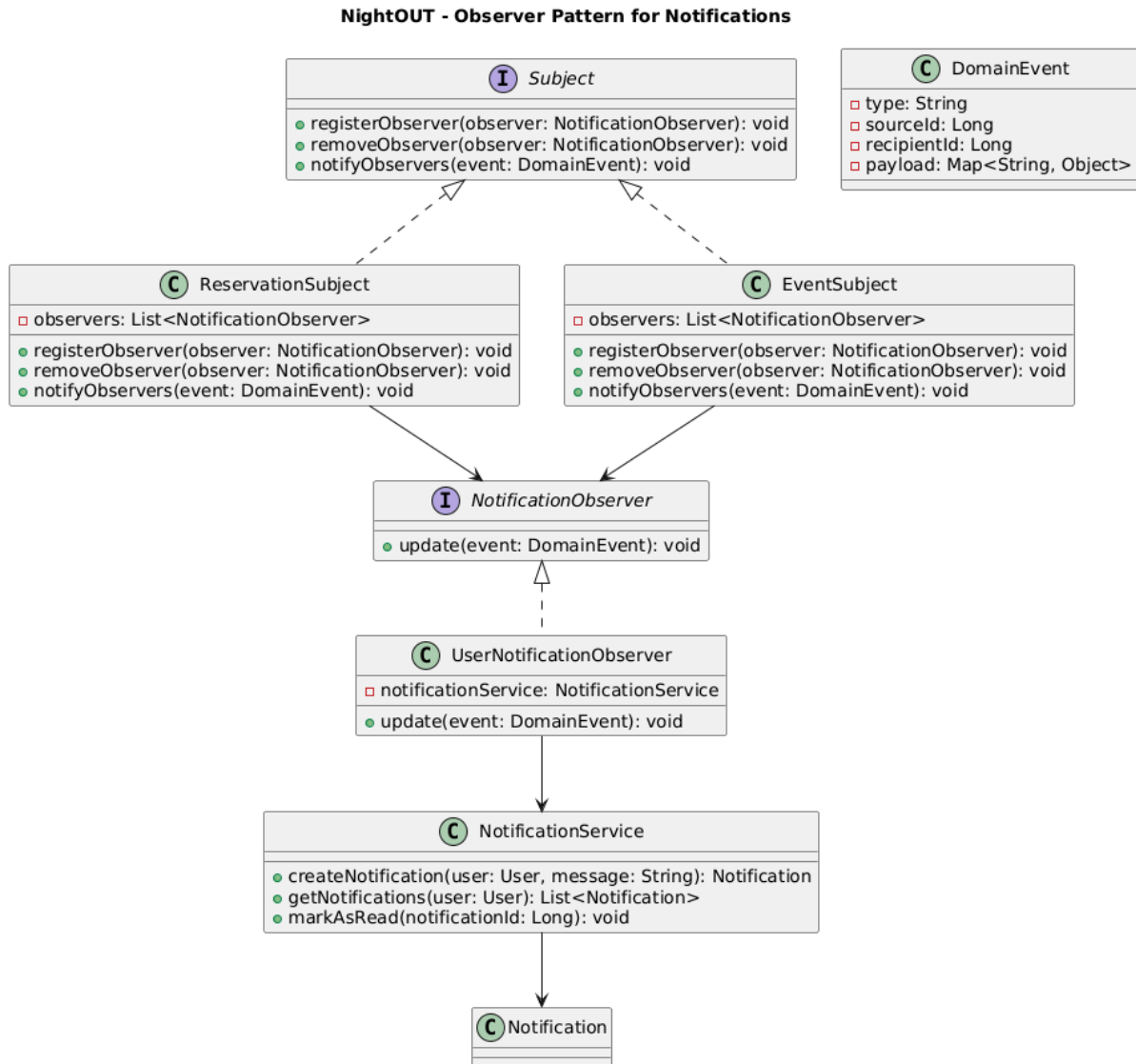
ReservationState <|.. PendingState
ReservationState <|.. ConfirmedState
ReservationState <|.. WaitingListState
ReservationState <|.. CancelledState
ReservationState <|.. ExpiredState

```

@enduml

2.3.3 Observer Pattern Class Diagram

The Observer pattern is used to decouple the components that generate relevant events from the components that create notifications.



@startuml

title NightOUT - Observer Pattern for Notifications

```

interface Subject {
    + registerObserver(observer: NotificationObserver): void
    + removeObserver(observer: NotificationObserver): void
    + notifyObservers(event: DomainEvent): void
}
    
```

```

interface NotificationObserver {
    + update(event: DomainEvent): void
}
    
```

```
}
```

```
class ReservationSubject {
```

```
- observers: List<NotificationObserver>
```

```
+ registerObserver(observer: NotificationObserver): void
```

```
+ removeObserver(observer: NotificationObserver): void
```

```
+ notifyObservers(event: DomainEvent): void
```

```
}
```

```
class EventSubject {
```

```
- observers: List<NotificationObserver>
```

```
+ registerObserver(observer: NotificationObserver): void
```

```
+ removeObserver(observer: NotificationObserver): void
```

```
+ notifyObservers(event: DomainEvent): void
```

```
}
```

```
class UserNotificationObserver {
```

```
- notificationService: NotificationService
```

```
+ update(event: DomainEvent): void
```

```
}
```

```
class NotificationService {
```

```
+ createNotification(user: User, message: String): Notification
```

```
+ getNotifications(user: User): List<Notification>
```

```
+ markAsRead(notificationId: Long): void
```

```
}
```

```
class DomainEvent {
```

```
- type: String
```

```
- sourceId: Long
```

```
- recipientId: Long
```

```
- payload: Map<String, Object>
```

```
}
```

```
Subject <|.. ReservationSubject
```

```
Subject <|.. EventSubject
```

```
NotificationObserver <|.. UserNotificationObserver
```

```
ReservationSubject --> NotificationObserver
```

```
EventSubject --> NotificationObserver
```

```
UserNotificationObserver --> NotificationService
```

```
NotificationService --> Notification
```

```
@enduml
```

2.3.4 Strategy Pattern for Filtering, Popularity and Recommendation



nota: questa immagine ha un problema e non ci sta nello schermo

@startuml

title NightOUT - Strategy Pattern for Filtering, Popularity and Recommendation

```
interface FilterStrategy {  
    + apply(events: List<Event>, criteria: FilterCriteria): List<Event>  
}
```

```
class DateFilterStrategy {  
    + apply(events: List<Event>, criteria: FilterCriteria): List<Event>  
}
```

```
class LocationFilterStrategy {  
    + apply(events: List<Event>, criteria: FilterCriteria): List<Event>  
}
```

```
class MusicGenreFilterStrategy {  
    + apply(events: List<Event>, criteria: FilterCriteria): List<Event>  
}
```

```
class PriceFilterStrategy {  
    + apply(events: List<Event>, criteria: FilterCriteria): List<Event>  
}
```

```
class EventSearchService {  
    - strategies: List<FilterStrategy>  
    + filterEvents(criteria: FilterCriteria): List<Event>  
}
```

```
interface PopularityStrategy {  
    + computeScore(event: Event): Double  
}
```

```
class MostSavedPopularityStrategy {  
    + computeScore(event: Event): Double  
}
```

```

class MostBookedPopularityStrategy {
+ computeScore(event: Event): Double
}

class FriendsAttendingPopularityStrategy {
+ computeScore(event: Event): Double
}

class PopularityService {
- strategy: PopularityStrategy
+ rankEvents(events: List<Event>): List<Event>
}

interface RecommendationStrategy {
+ recommend(user: User, events: List<Event>): List<Event>
}

class MusicPreferenceRecommendationStrategy {
+ recommend(user: User, events: List<Event>): List<Event>
}

class SocialProximityRecommendationStrategy {
+ recommend(user: User, events: List<Event>): List<Event>
}

class DistanceBasedRecommendationStrategy {
+ recommend(user: User, events: List<Event>): List<Event>
}

class BookingHistoryRecommendationStrategy {
+ recommend(user: User, events: List<Event>): List<Event>
}

class RecommendationService {
- strategy: RecommendationStrategy
+ recommendEvents(user: User): List<Event>
}

FilterStrategy <|.. DateFilterStrategy
FilterStrategy <|.. LocationFilterStrategy
FilterStrategy <|.. MusicGenreFilterStrategy
FilterStrategy <|.. PriceFilterStrategy

EventSearchService --> FilterStrategy

PopularityStrategy <|.. MostSavedPopularityStrategy
PopularityStrategy <|.. MostBookedPopularityStrategy

```

PopularityStrategy <|.. FriendsAttendingPopularityStrategy

PopularityService --> PopularityStrategy

RecommendationStrategy <|.. MusicPreferenceRecommendationStrategy

RecommendationStrategy <|.. SocialProximityRecommendationStrategy

RecommendationStrategy <|.. DistanceBasedRecommendationStrategy

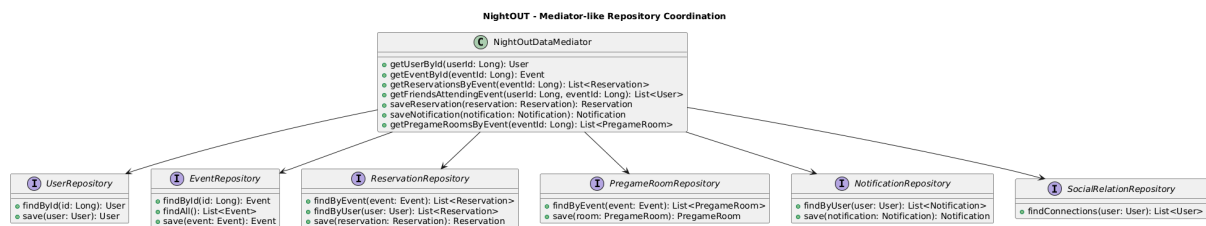
RecommendationStrategy <|.. BookingHistoryRecommendationStrategy

RecommendationService --> RecommendationStrategy

@enduml

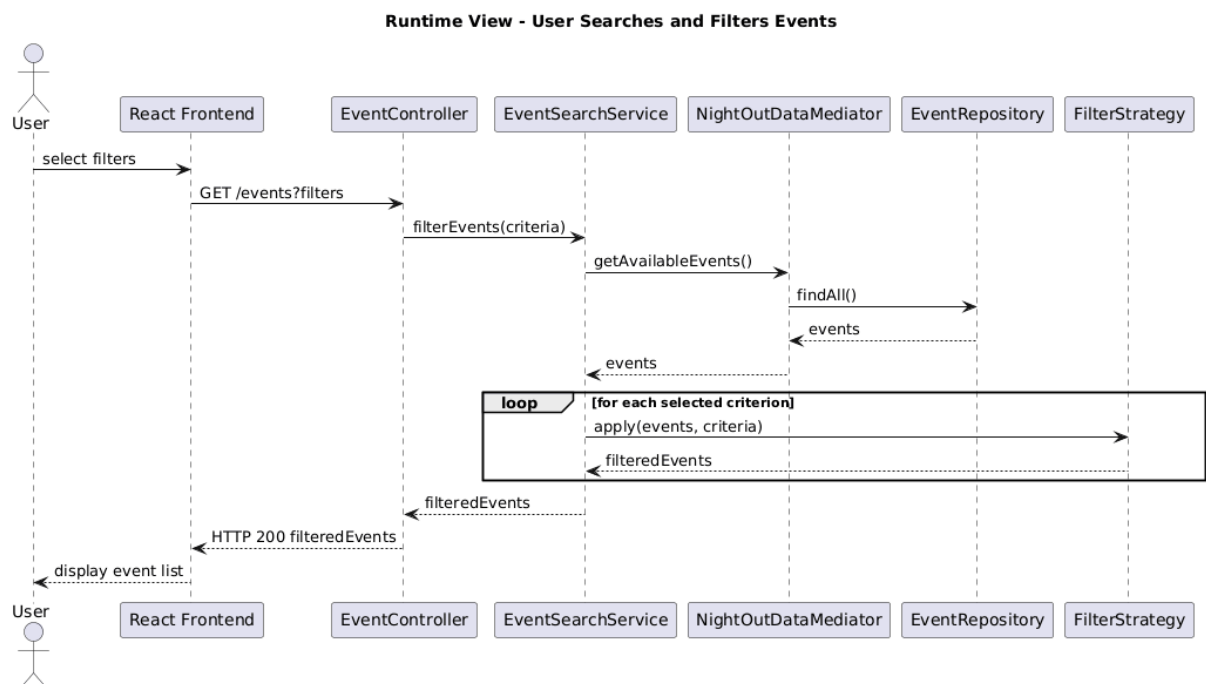
2.3.5 Mediator-like Repository Coordination

The system uses a mediator-like component to coordinate data access across repositories. This avoids direct dependencies between high-level services and multiple repositories.



2.4 Runtime View

2.4.1 User Searches and Filters Events



```

@startuml
title Runtime View - User Searches and Filters Events

actor User
participant "React Frontend" as Frontend
participant "EventController" as EventController
participant "EventSearchService" as SearchService
participant "NightOutDataMediator" as Mediator
participant "EventRepository" as EventRepository
participant "FilterStrategy" as FilterStrategy

User -> Frontend : select filters
Frontend -> EventController : GET /events?filters
EventController -> SearchService : filterEvents(criteria)
SearchService -> Mediator : getAvailableEvents()
Mediator -> EventRepository : findAll()
EventRepository --> Mediator : events
Mediator --> SearchService : events

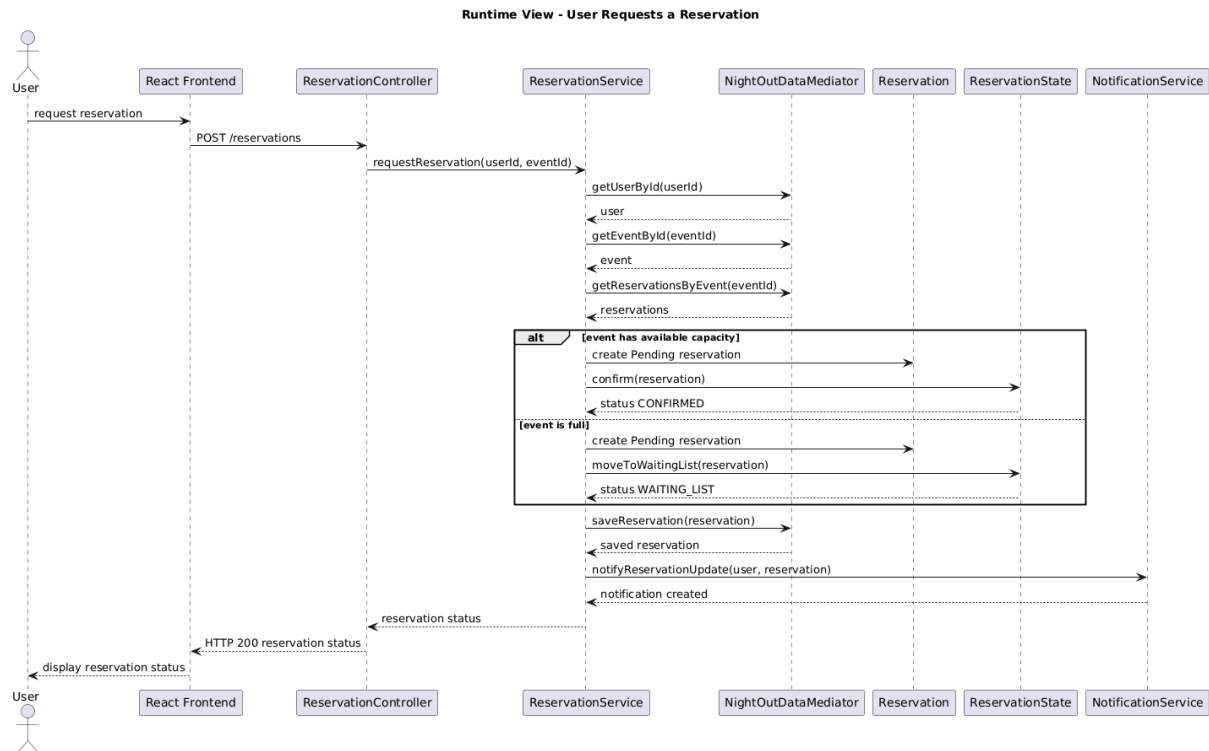
loop for each selected criterion
    SearchService -> FilterStrategy : apply(events, criteria)
    FilterStrategy --> SearchService : filteredEvents
end

SearchService --> EventController : filteredEvents
EventController --> Frontend : HTTP 200 filteredEvents
Frontend --> User : display event list

@enduml

```

2.4.2 User Requests a Reservation



@startuml

title Runtime View - User Requests a Reservation

actor User

participant "React Frontend" as Frontend

participant "ReservationController" as ReservationController

participant "ReservationService" as ReservationService

participant "NightOutDataMediator" as Mediator

participant "Reservation" as Reservation

participant "ReservationState" as ReservationState

participant "NotificationService" as NotificationService

User -> Frontend : request reservation

Frontend -> ReservationController : POST /reservations

ReservationController -> ReservationService : requestReservation(userId, eventId)

ReservationService -> Mediator : getUserById(userId)

Mediator --> ReservationService : user

ReservationService -> Mediator : getEventById(eventId)

Mediator --> ReservationService : event

ReservationService -> Mediator : getReservationsByEvent(eventId)

Mediator --> ReservationService : reservations

alt event has available capacity

ReservationService -> Reservation : create Pending reservation

```

ReservationService -> ReservationState : confirm(reservation)
ReservationState --> ReservationService : status CONFIRMED
else event is full
  ReservationService -> Reservation : create Pending reservation
  ReservationService -> ReservationState : moveToWaitingList(reservation)
  ReservationState --> ReservationService : status WAITING_LIST
end

ReservationService -> Mediator : saveReservation(reservation)
Mediator --> ReservationService : saved reservation

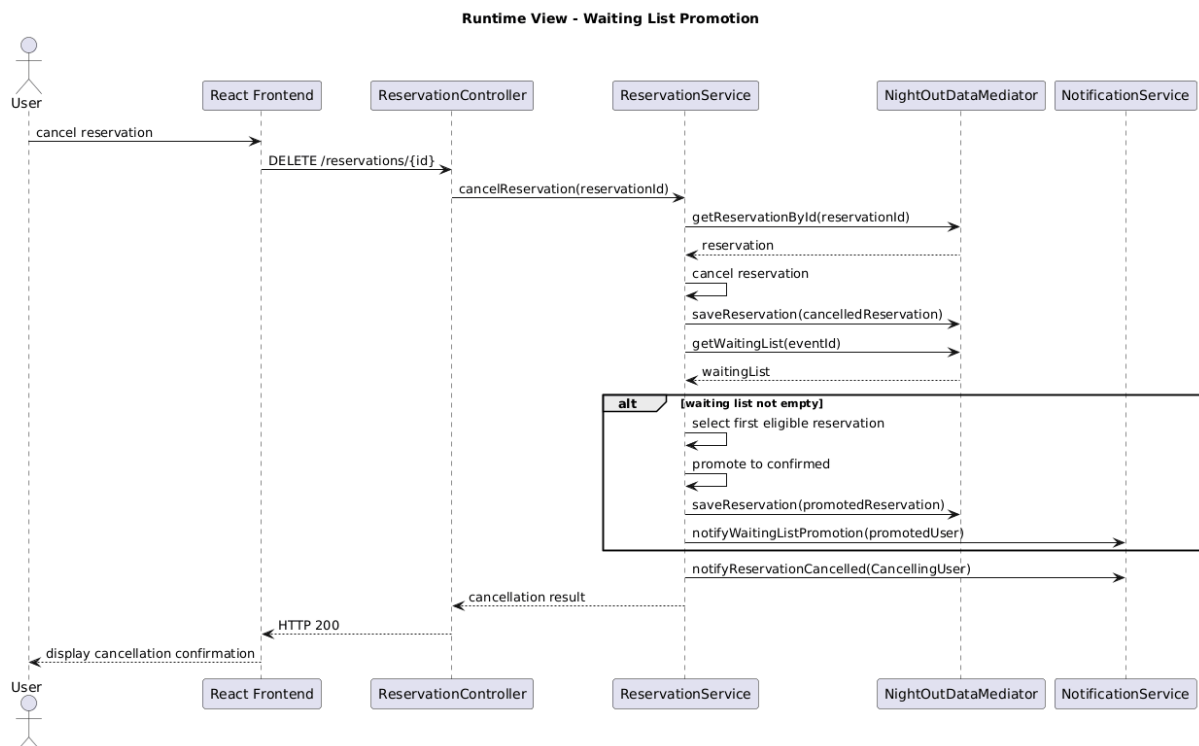
ReservationService -> NotificationService : notifyReservationUpdate(user, reservation)
NotificationService --> ReservationService : notification created

ReservationService --> ReservationController : reservation status
ReservationController --> Frontend : HTTP 200 reservation status
Frontend --> User : display reservation status

```

@enduml

2.4.3 Waiting List Promotion After Cancellation



@startuml

title Runtime View - Waiting List Promotion

actor User as CancellingUser

```

participant "React Frontend" as Frontend
participant "ReservationController" as ReservationController
participant "ReservationService" as ReservationService
participant "NightOutDataMediator" as Mediator
participant "NotificationService" as NotificationService

CancellingUser -> Frontend : cancel reservation
Frontend -> ReservationController : DELETE /reservations/{id}
ReservationController -> ReservationService : cancelReservation(reservationId)

ReservationService -> Mediator : getReservationById(reservationId)
Mediator --> ReservationService : reservation

ReservationService -> ReservationService : cancel reservation
ReservationService -> Mediator : saveReservation(cancelledReservation)

ReservationService -> Mediator : getWaitingList(eventId)
Mediator --> ReservationService : waitingList

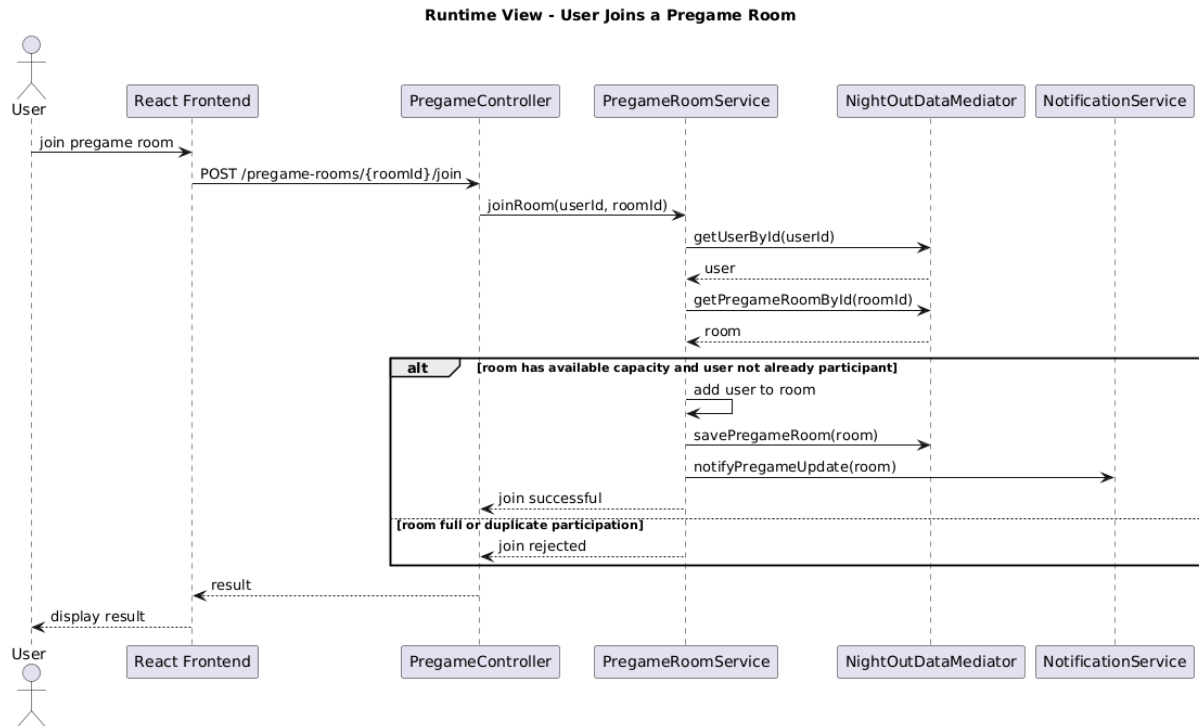
alt waiting list not empty
    ReservationService -> ReservationService : select first eligible reservation
    ReservationService -> ReservationService : promote to confirmed
    ReservationService -> Mediator : saveReservation(promotedReservation)
    ReservationService -> NotificationService : notifyWaitingListPromotion(promotedUser)
end

ReservationService -> NotificationService : notifyReservationCancelled(CancellingUser)
ReservationService --> ReservationController : cancellation result
ReservationController --> Frontend : HTTP 200
Frontend --> CancellingUser : display cancellation confirmation

@enduml

```

2.4.4 User Joins a Pregame Room



@startuml

title Runtime View - User Joins a Pregame Room

actor User

participant "React Frontend" as Frontend

participant "PregameController" as PregameController

participant "PregameRoomService" as PregameService

participant "NightOutDataMediator" as Mediator

participant "NotificationService" as NotificationService

User -> Frontend : join pregame room

Frontend -> PregameController : POST /pregame-rooms/{roomId}/join

PregameController -> PregameService : joinRoom(userId, roomId)

PregameService -> Mediator : getUserById(userId)

Mediator --> PregameService : user

PregameService -> Mediator : getPregameRoomById(roomId)

Mediator --> PregameService : room

alt room has available capacity and user not already participant

 PregameService -> PregameService : add user to room

 PregameService -> Mediator : savePregameRoom(room)

 PregameService -> NotificationService : notifyPregameUpdate(room)

 PregameService --> PregameController : join successful

else room full or duplicate participation

 PregameService --> PregameController : join rejected

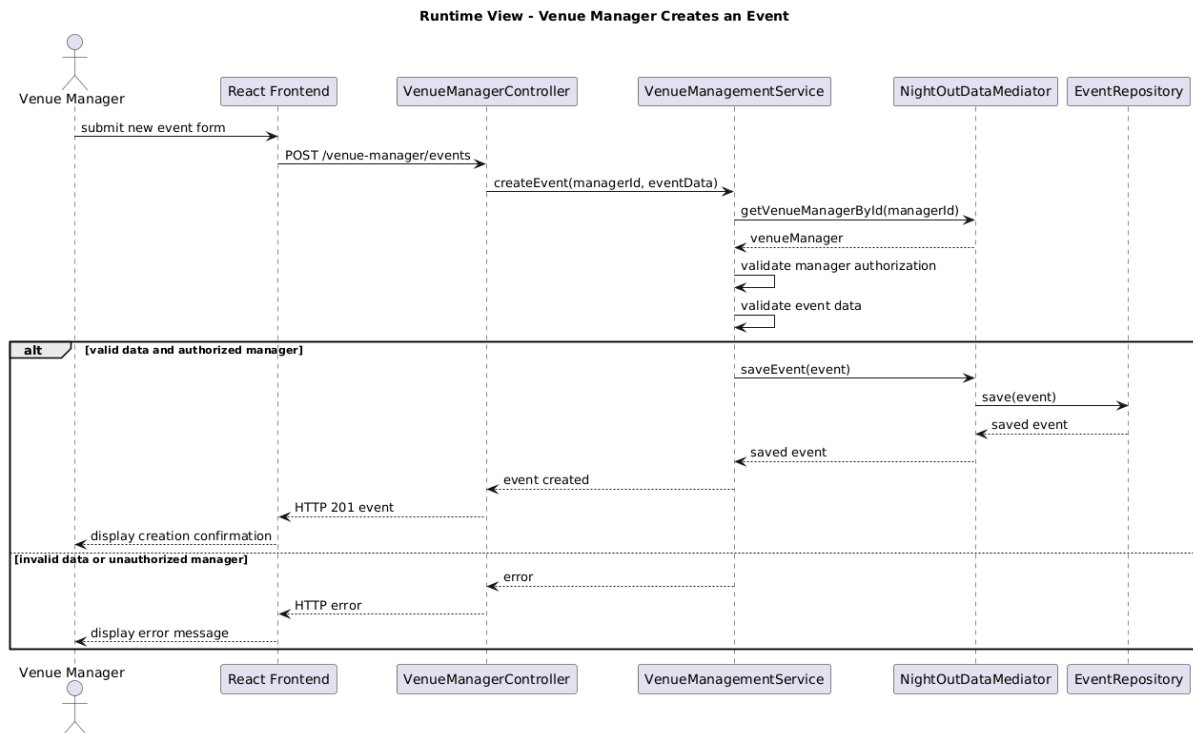
end

PregameController --> Frontend : result

Frontend --> User : display result

@enduml

2.4.5 Venue Manager Creates an Event



@startuml

title Runtime View - Venue Manager Creates an Event

actor "Venue Manager" as VM

participant "React Frontend" as Frontend

participant "VenueManagerController" as Controller

participant "VenueManagementService" as VenueService

participant "NightOutDataMediator" as Mediator

participant "EventRepository" as EventRepository

VM -> Frontend : submit new event form

Frontend -> Controller : POST /venue-manager/events

Controller -> VenueService : createEvent(managerId, eventData)

VenueService -> Mediator : getVenueManagerById(managerId)

Mediator --> VenueService : venueManager

VenueService -> VenueService : validate manager authorization

```
VenueService -> VenueService : validate event data
```

```
alt valid data and authorized manager
```

```
    VenueService -> Mediator : saveEvent(event)
```

```
    Mediator -> EventRepository : save(event)
```

```
    EventRepository --> Mediator : saved event
```

```
    Mediator --> VenueService : saved event
```

```
    VenueService --> Controller : event created
```

```
    Controller --> Frontend : HTTP 201 event
```

```
    Frontend --> VM : display creation confirmation
```

```
else invalid data or unauthorized manager
```

```
    VenueService --> Controller : error
```

```
    Controller --> Frontend : HTTP error
```

```
    Frontend --> VM : display error message
```

```
end
```

```
@enduml
```

2.5 Selected Architectural Styles and Patterns

2.5.1 Client-Server Architecture

NightOUT uses a client-server architecture. The React frontend acts as the client and communicates with the Java Spring Boot backend through REST APIs. The backend provides services and manages persistence.

This choice allows the frontend and backend to evolve independently and supports a clear separation between presentation logic and business logic.

2.5.2 Layered Architecture

The backend follows a layered architecture:

- controllers expose REST endpoints;
- services implement application logic;
- domain classes represent business entities and behavior;
- repositories provide data access;
- the database stores persistent data.

This architecture improves maintainability, testability, and separation of concerns.

2.5.3 State Pattern

The State pattern is used in the Reservation and Ticketing module.

A reservation can evolve through several states:

- pending;
- confirmed;
- waiting list;
- cancelled;
- expired.

The operations allowed on a reservation depend on its current state. For example, a waiting list reservation can become confirmed when a spot becomes available, while a cancelled reservation cannot become confirmed again.

The State pattern avoids concentrating all transition logic inside a single monolithic class. Instead, each state is represented by a dedicated class implementing the same interface.

2.5.4 Observer Pattern

The Observer pattern is used in the Notification component.

Relevant domain events include:

- reservation status changes;
- waiting list promotion;
- friend joining an event;
- event updates;
- pregame room updates.

The components that generate events do not need to know how notifications are delivered. Instead, observers react to domain events and create user notifications. This reduces coupling and makes it easier to add new notification types.

2.5.5 Strategy Pattern

The Strategy pattern is used in three main areas:

1. **Event filtering**
2. **Event popularity computation**
3. **Event recommendation**

Each of these areas requires interchangeable algorithms. For example, event filtering may use date, location, genre, or price filters. Popularity may be computed based on saved events, bookings, or friends attending. Recommendation may be based on music preferences, social proximity, distance, or booking history.

The Strategy pattern allows the system to introduce new algorithms without modifying the services that use them.

2.5.6 Mediator-like Repository Coordination

NightOUT introduces a mediator-like data access component called `NightOutDataMediator`.

This component coordinates repositories and supports complex operations involving multiple domain entities. For example, generating recommendations may require user preferences, social relations, reservations, and event information. Without a mediator-like component, high-level services would directly depend on many repositories.

This design reduces coupling and centralizes complex data access operations.

2.5.7 Patterns Intentionally Not Used

Some additional patterns are intentionally not used in the prototype.

The Adapter pattern is not used because the prototype does not integrate external APIs such as real ticketing systems, map services, or payment providers.

The Singleton pattern is not explicitly implemented because Spring Boot already manages service and repository instances through its application context. Introducing manual global access points would increase coupling and reduce testability.

3. User Interface Design

3.1 UI Overview

NightOUT is designed as a mobile-oriented web application. The interface is optimized for smartphone usage and focuses on quick access to event discovery, reservation, pregame coordination, and social information.

The main design principles are:

- simple navigation;
- event-first layout;
- clear display of event information;
- fast reservation flow;
- mobile-friendly cards and buttons;
- separation between user interface and venue manager interface.

The frontend communicates with the backend through REST API calls.

3.2 Main User Interfaces

3.2.1 Home / Event Feed

The home screen displays a feed of suggested nightlife events.

Main elements:

- event cards;
- event title;
- venue name;
- date and time;
- music genre;
- price or entry condition;
- popularity indicators;
- quick access to event details.

Main actions:

- browse events;
- open event details;
- save event;
- apply filters.

3.2.2 Event Search and Filters

The search and filtering screen allows users to filter events.

Available filters:

- date;
- location or area;
- music genre;
- venue category;
- price or entry condition.

The result is shown as a list of event cards. If no event satisfies the filters, the system displays an empty result message.

3.2.3 Event Detail Page

The event detail page displays all relevant information about a selected event.

Main information:

- event title;
- venue;
- location;
- date and time;
- music genre;
- dress code;
- age target;
- price or entry condition;
- promotions;
- capacity or availability status;

- friends attending or interested;
- linked pregame rooms.

Main actions:

- reserve a spot;
- join waiting list;
- save event;
- view pregame rooms;
- create or join pregame room.

3.2.4 Reservation / Ticket Section

The reservation section allows users to view their reservations.

Main information:

- event;
- date and time;
- reservation status;
- digital confirmation;
- waiting list status, if applicable.

Main actions:

- view reservation;
- cancel reservation.

3.2.5 Pregame Room Page

The pregame room page allows users to browse, create, join, and leave pregame rooms.

Main information:

- linked event;
- host;
- meeting location;
- meeting time;
- maximum number of participants;
- current participants;
- description.

Main actions:

- create room;
- join room;
- leave room.

3.2.6 Social / Activity Page

The social page displays relevant event-related activity from friends or followed users.

Main information:

- friends attending events;
- friends saving events;
- popular events among connections;
- suggested events based on social proximity.

3.2.7 Notification Page

The notification page displays user notifications.

Notification types:

- reservation updates;
- waiting list promotion;
- friend joined event;
- event update;
- pregame update.

Main actions:

- view notifications;
- mark notifications as read.

3.2.8 User Profile Page

The user profile page displays user information and user-related event data.

Main information:

- profile data;
- music preferences;
- saved events;
- reservations;
- pregame rooms joined or hosted;
- social connections.

3.3 Venue Manager Interface

Venue managers access a specific dashboard.

3.3.1 Venue Manager Dashboard

The dashboard displays venues and events managed by the venue manager.

Main information:

- managed venues;
- published events;
- upcoming events;
- basic event statistics.

3.3.2 Event Creation and Editing Page

This page allows venue managers to create or update events.

Required fields:

- title;
- venue;
- date and time;
- music genre;
- dress code;
- price or entry condition;
- capacity;
- description;
- promotions.

3.3.3 Event Statistics Page

This page displays basic statistics about events.

Main indicators:

- number of saved events;
- number of confirmed reservations;
- waiting list size;
- number of pregame rooms linked to the event;
- popularity score.

4. Requirements Traceability

4.1 Traceability Matrix

The following table maps the main requirements defined in the Requirements Analysis and Specification Document to the corresponding design elements.

Requirement ID	Requirement Summary	Design Elements
FR-US-01	User registration	AuthController, AuthenticationService, UserRepository, User

FR-US-02	User login	AuthController, AuthenticationService
FR-US-03	View user profile	UserController, UserService, UserRepository, User Profile UI
FR-US-04	Define music preferences	UserService, User, User Profile UI
FR-US-05	Save event	EventController, UserService, EventParticipation, EventRepository
FR-US-06	View saved events	UserController, UserService, EventParticipation
FR-EV-01	Browse events	EventController, EventService, EventRepository, Event Feed UI
FR-EV-02	View event details	EventController, EventService, Event Detail UI
FR-EV-03	Display event information	Event, Venue, Event Detail UI
FR-EV-04	Filter events	EventSearchService, FilterStrategy, EventController
FR-EV-05	Empty filter result	EventSearchService, Event Search UI
FR-EV-06	Popularity view	PopularityService, PopularityStrategy, Event Popularity UI
FR-REC-01	Recommended event feed	RecommendationService, RecommendationStrategy, Event Feed UI
FR-REC-02	Music-based recommendation	MusicPreferenceRecommendationStrategy
FR-REC-03	Social-based recommendation	SocialProximityRecommendationStrategy, SocialService
FR-REC-04	Distance-based recommendation	DistanceBasedRecommendationStrategy
FR-REC-05	Booking-history recommendation	BookingHistoryRecommendationStrategy, ReservationRepository
FR-RS-01	Request reservation	ReservationController, ReservationService
FR-RS-02	Confirm reservation with capacity	ReservationService, Reservation, ConfirmedState
FR-RS-03	Waiting list for full events	ReservationService, WaitingListState

FR-RS-04	View reservation status	ReservationController, ReservationService, Ticket Section UI
FR-RS-05	Cancel reservation	ReservationService, ReservationState, CancelledState
FR-RS-06	Promote waiting list reservation	ReservationService, WaitingListState, NotificationService
FR-RS-07	Prevent duplicate active reservations	ReservationService, ReservationRepository
FR-RS-08	Reservation status values	ReservationStatus enum, ReservationState hierarchy
FR-RS-09	Digital reservation confirmation	Reservation, Ticket Section UI
FR-PR-01	Create pregame room	PregameController, PregameRoomService, PregameRoom
FR-PR-02	Store pregame room information	PregameRoom, PregameRoomRepository
FR-PR-03	Browse pregame rooms	PregameController, PregameRoomService, Pregame Room UI
FR-PR-04	Join pregame room	PregameRoomService, PregameRoom
FR-PR-05	Prevent joining full room	PregameRoomService, PregameRoom
FR-PR-06	Leave pregame room	PregameRoomService
FR-PR-07	Prevent duplicate participation	PregameRoomService, PregameRoomRepository
FR-SO-01	Create social relation	SocialService, SocialRelationRepository
FR-SO-02	View social connections	SocialService, Social UI
FR-SO-03	Event participation status	EventParticipation, SocialService
FR-SO-04	Friends attending event	SocialService, Event Detail UI
FR-SO-05	Activity feed	SocialService, Activity Feed UI
FR-NO-01	Reservation notification	NotificationService, Observer Pattern

FR-NO-02	Waiting list notification	NotificationService, Observer Pattern
FR-NO-03	Friend event notification	NotificationService, SocialService, Observer Pattern
FR-NO-04	Event update notification	NotificationService, EventService
FR-NO-05	View notifications	NotificationController, NotificationService, Notification UI
FR-NO-06	Mark notification as read	NotificationController, NotificationService, Notification
FR-VM-01	Venue manager login	AuthController, AuthenticationService
FR-VM-02	Create event	VenueManagerController, VenueManagementService, EventRepository
FR-VM-03	Update event	VenueManagementService, EventRepository
FR-VM-04	Manage event capacity	VenueManagementService, Event
FR-VM-05	Create promotion	VenueManagementService, Promotion
FR-VM-06	View event statistics	VenueManagementService, PopularityService, Venue Manager Dashboard

4.2 Traceability Discussion

The traceability matrix shows that each functional requirement is mapped to one or more design elements.

Reservation requirements are mainly mapped to the Reservation and Ticketing component and to the State pattern hierarchy. This ensures that the reservation lifecycle specified in the requirements is directly reflected in the design.

Notification requirements are mapped to the Notification component and to the Observer pattern. This allows the system to generate notifications in response to reservation, event, social, and pregame updates.

Filtering, popularity, and recommendation requirements are mapped to services and strategy interfaces. This supports extensibility because new filtering criteria, popularity metrics, or recommendation algorithms can be introduced without modifying the core services.

Venue manager requirements are mapped to the Venue Management component, which handles event creation, event updates, promotion management, and event statistics.

Pregame requirements are mapped to the Pregame Room component, which manages room creation, capacity, participation, and event linkage.

5. Implementation, Integration and Test Plan

5.1 Implementation Order

The implementation will follow an incremental approach. Features are ordered according to dependency and complexity.

Step 1 — User and Authentication Management

Implemented first because most other features require registered users.

Main elements:

- User;
- VenueManager;
- AuthenticationService;
- UserService;
- AuthController;
- UserController.

Step 2 — Venue and Event Management

Implemented after user management because events are the central domain objects of NightOUT.

Main elements:

- Venue;
- Event;
- VenueManagementService;
- EventService;
- EventController;
- VenueManagerController.

Step 3 — Event Browsing, Search and Filtering

Implemented after events are available in the system.

Main elements:

- EventSearchService;
- FilterStrategy;

- DateFilterStrategy;
- LocationFilterStrategy;
- MusicGenreFilterStrategy;
- PriceFilterStrategy.

Step 4 — Reservation and Ticketing Module

Implemented after event management because reservations depend on events and capacity.

Main elements:

- Reservation;
- ReservationStatus;
- ReservationState;
- PendingState;
- ConfirmedState;
- WaitingListState;
- CancelledState;
- ExpiredState;
- ReservationService;
- ReservationController.

Step 5 — Basic Notification System

Implemented after reservation management because the first notification scenarios are reservation status changes and waiting list promotions.

Main elements:

- Notification;
- NotificationService;
- NotificationController;
- NotificationObserver;
- ReservationSubject.

Step 6 — Pregame Room Management

Implemented after users and events are available.

Main elements:

- PregameRoom;
- PregameRoomService;
- PregameController.

Step 7 — Social Layer

Implemented after events and users are available.

Main elements:

- SocialRelation;
- EventParticipation;
- SocialService.

Step 8 — Event Popularity Dashboard

Implemented after reservations, saved events, and social data are available.

Main elements:

- PopularityService;
- PopularityStrategy;
- MostSavedPopularityStrategy;
- MostBookedPopularityStrategy;
- FriendsAttendingPopularityStrategy.

Step 9 — Recommendation Engine

Implemented last because it depends on user preferences, event data, social relations, distance, and booking history.

Main elements:

- RecommendationService;
- RecommendationStrategy;
- MusicPreferenceRecommendationStrategy;
- SocialProximityRecommendationStrategy;
- DistanceBasedRecommendationStrategy;
- BookingHistoryRecommendationStrategy.

5.2 Integration Plan

The integration plan follows the same incremental order.

Integration 1 — Backend Domain and Repositories

Domain classes are integrated with repository interfaces and database persistence.

Integration 2 — Services and Repositories

Application services are connected to repositories through the [NightOutDataMediator](#).

Integration 3 — REST Controllers and Services

REST controllers are connected to services and tested through HTTP requests.

Integration 4 — Frontend and Backend

The React frontend is connected to backend REST APIs.

Integration 5 — Reservation and Notification Integration

Reservation state changes are integrated with notification generation.

Integration 6 — Pregame and Event Integration

Pregame rooms are integrated with event detail pages.

Integration 7 — Social, Popularity and Recommendation Integration

Social data is integrated with popularity and recommendation services.

5.3 Testing Strategy

The testing strategy includes unit testing, integration testing, black-box testing, white-box testing, boundary value testing, and end-to-end scenario testing.

Testing is used to verify that the implementation conforms to the requirements and design specifications. It cannot prove the absence of defects, but it can expose failures and support debugging.

5.3.1 Unit Testing

Unit tests will focus on backend services and domain logic.

Main classes to test:

- ReservationService;
- ReservationState implementations;
- EventService;
- EventSearchService;
- PregameRoomService;
- NotificationService;
- PopularityService;
- RecommendationService.

Example unit test groups:

- reservation with available capacity;
- reservation for full event;
- waiting list promotion;
- duplicate reservation prevention;
- pregame room join;
- pregame room full;

- filtering by genre, date, and location;
- notification creation.

5.3.2 Black-Box Testing

Black-box tests are derived from functional requirements.

Examples:

Requirement	Test Case
Filter events	Filter by date, genre, location, and price
Reserve event	Request reservation for event with capacity
Waiting list	Request reservation for full event
Join pregame room	Join room with available capacity
Prevent full room join	Try to join a full pregame room
Venue manager creates event	Submit valid event data
Invalid event creation	Submit missing required fields

5.3.3 White-Box Testing

White-box tests are applied mainly to backend service methods. The goal is to exercise relevant branches and conditions.

Examples:

- available capacity branch vs full event branch in `ReservationService`;
- valid vs invalid reservation request;
- full vs available pregame room;
- valid vs invalid venue manager authorization;
- each concrete filtering strategy;
- each reservation state transition.

The project will target meaningful statement and branch coverage for core backend services.

5.3.4 Boundary Value Testing

Boundary cases are tested because defects often occur at the edges of valid input ranges.

Examples:

Feature	Boundary Cases
Event capacity	capacity = 0, capacity = 1, confirmed reservations = capacity
Reservation	duplicate active reservation, cancellation of already cancelled reservation
Pregame room	maxParticipants = 1, currentParticipants = maxParticipants
Filtering	no result, one result, all events returned
Event creation	empty title, missing venue, past date
Price filter	free event, maximum price value

5.3.5 Integration Testing

Integration tests verify that components work correctly together.

Examples:

- ReservationService + ReservationRepository;
- ReservationService + NotificationService;
- EventService + EventRepository;
- PregameRoomService + EventRepository;
- SocialService + RecommendationService;
- VenueManagementService + EventRepository.

5.3.6 End-to-End Testing

End-to-end tests follow the main user scenarios.

E2E-01 — User Discovers and Reserves an Event

1. The user opens the event feed.
2. The user filters events.
3. The user opens an event detail page.
4. The user requests a reservation.
5. The system displays the reservation status.

E2E-02 — Full Event and Waiting List

1. The user opens a full event.
2. The user requests a reservation.
3. The system places the user in the waiting list.
4. Another reservation is cancelled.
5. The system promotes the waiting list reservation.
6. The user receives a notification.

E2E-03 — Pregame Room

1. The user opens an event detail page.
2. The user browses linked pregame rooms.
3. The user joins a room.
4. The system updates the participant list.

E2E-04 — Venue Manager Creates an Event

1. The venue manager logs in.
2. The venue manager creates an event.
3. The system stores the event.
4. The event becomes visible in the user feed.

5.4 Test Data

The prototype uses a realistic synthetic dataset generated by the development team.

The dataset includes:

- users;
- venue managers;
- venues;
- events;
- reservations;
- waiting list entries;
- social relations;
- saved events;
- event participation data;
- pregame rooms;
- promotions;
- notifications.

The dataset is designed not only to be realistic, but also to cover important test scenarios.

Required dataset cases:

- events with available capacity;
- full events;
- events with waiting lists;
- users with existing reservations;
- users with duplicate reservation attempts;
- pregame rooms with available capacity;
- full pregame rooms;
- social relations between users;
- events attended by friends;
- events with different genres, locations, and prices.

A possible initial dataset size is:

Entity	Approximate Number
Users	40
Venue Managers	8
Venues	12
Events	50
Reservations	120
Saved Events	80
Social Relations	70
Pregame Rooms	15
Promotions	20
Notifications	30

6. References

1. SE4A — Modular Design and Software Architecture slides.
2. SE4A — Object Orientation and Software Architecture slides.
3. SE4A — Design Patterns slides.
4. SE4A — Object Orientation: Behavioral Views slides.
5. SE4A — Requirements Engineering slides.
6. SE4A — Verification and Validation, Testing slides.

7. INTRODUCTION

- a. Purpose: recap of the main goals of the project
- b. Definitions, Acronyms, Abbreviations.
- c. Revision history.
- d. Document Structure

8. ARCHITECTURAL DESIGN

- a. Component view: component diagram, brief description of components and main operations exported by interfaces.
- b. Class view: further details on individual components through class diagrams (if relevant).
- c. Runtime view: sequence diagrams to describe the way components (or objects) interact to accomplish specific tasks typically related to scenarios/requirements.
- d. Selected architectural styles and patterns: Please explain which styles/patterns you used, why, and how

9. USER INTERFACE DESIGN

Provide an overview on how the user interface(s) of your system will look like.

10. REQUIREMENTS TRACEABILITY

show and explain how the requirements you have defined in the requirements analysis document map to the design elements defined in this document

11. IMPLEMENTATION, INTEGRATION AND TEST PLAN

Identify here the order in which you plan to implement the subcomponents of your system and the order in which you plan to integrate such subcomponents and test the integration.

12. REFERENCES

Design

- Defines the **software architecture**
 - Components (modules)
 - Relations among components
 - Interactions among components
- Goal
 - Support (concurrent) development, separate responsibilities
- **Outcome**
 - **Design Document (DD)**